

Aula 12 - Acessibilidade web

Descrição da Aula: Nesta aula, vamos mergulhar no universo da acessibilidade digital, compreendendo como projetar interfaces inclusivas seguindo as diretrizes WCAG 2.1. Exploraremos a utilização prática de atributos **ARIA**, o correto emprego da tag `alt` em imagens e a garantia de uma **navegação fluida** via teclado. A aula adota uma abordagem prática de auditoria utilizando a ferramenta **Lighthouse (DevTools)**, culminando em um **code review** orientado por um *checklist* de acessibilidade, onde cada equipe deverá corrigir ao menos três problemas identificados em seus projetos.

Entregas da aula: Relatório Lighthouse + correções aplicadas no projeto.

1. Abertura (Contexto da aula)

Até o momento, vocês construíram páginas incríveis e responsivas para o produto digital da nossa Situação de Aprendizagem. Porém, **um bom produto tecnológico não é apenas visualmente agradável**; ele precisa **ser utilizável por todas as pessoas**, independentemente de suas limitações **físicas, motoras ou cognitivas**. Atender a requisitos de qualidade e usabilidade é uma capacidade técnica fundamental. Hoje, deixaremos o **design estético** um pouco de lado para focar na engenharia por trás da **inclusão digital**. Garantir acessibilidade é uma exigência legal e ética no mercado atual.

2. Conceitos Fundamentais e Base Técnica

Conceito 1: Diretrizes WCAG 2.1 e Navegação por Teclado

A. Pergunta: Se o seu **mouse parasse de funcionar** agora, você conseguiria navegar, preencher um formulário e finalizar uma compra no seu site apenas utilizando a tecla `Tab` e `Enter`?

B. Cenário Real:

Imagine um usuário com **deficiência motora** severa que utiliza um **head pointer** (ponteiro de cabeça) e um **teclado adaptado** para navegar na web. Se o seu site não possuir foco visível ou se os botões não puderem ser acessados sequencialmente, esse usuário ficará bloqueado na primeira página e abandonará a plataforma, gerando perda de clientes e possível passivo jurídico para a empresa.

C. Explicação teórica:

A acessibilidade na web é regida por um **conjunto internacional de diretrizes** chamado **WCAG (Web Content Accessibility Guidelines)**, atualmente na **versão 2.1**. Essas diretrizes baseiam-se em quatro princípios fundamentais, conhecidos pelo acrônimo **POUR**: o conteúdo deve ser **Perceptível** (Informação disponível aos sentidos), **Operável** (Interface navegável), **Compreensível** (Conteúdo claro) e **Robusto** (Funciona em diversas tecnologias - leitores de tela). Compreender esses princípios é o primeiro passo para o desenvolvimento de sistemas alinhados aos padrões globais de qualidade e usabilidade.

Perceptível (Perceivable)

Os usuários devem ser capazes de perceber as informações e os elementos da interface. Nada pode ser "invisível" a todos os sentidos do usuário.

Na prática: Se você tem uma imagem importante, um usuário cego não pode vê-la. Para torná-la perceptível, você adiciona um texto alternativo (`alt`) que um leitor de tela (como o NVDA ou VoiceOver) lerá em voz alta. Outro exemplo é fornecer legendas para vídeos, beneficiando pessoas surdas.

Operável (Operable)

Os componentes da interface e a navegação devem ser operáveis. A interface não pode exigir interações que o usuário não consiga realizar.

Na prática: Muitas pessoas com deficiências motoras não conseguem usar um mouse, dependendo exclusivamente do teclado (ou de dispositivos adaptados) para navegar. Um site operável permite que todos os botões, links e formulários sejam acessados e ativados apenas usando as teclas Tab, Enter e Espaço, sem criar "armadilhas" onde o cursor fica preso.

Compreensível (Understandable)

A informação e a operação da interface devem ser lógicas e claras.

Na prática: Isso envolve usar uma linguagem simples e direta, evitar jargões desnecessários e garantir que o site se comporte de maneira previsível. Por exemplo, se um usuário preencher um formulário com erro, o site deve explicar claramente onde está o erro e como corrigi-lo, em vez de apenas piscar uma borda vermelha.

Robusto (Robust)

O conteúdo deve ser robusto o suficiente para ser interpretado de forma confiável por uma ampla variedade de navegadores e tecnologias assistivas (softwares que ajudam pessoas com deficiência a usar o computador).

Na prática: Isso significa escrever um código HTML semântico e limpo. Usar a tag `<button>` para botões e `<nav>` para navegação, em vez de criar tudo usando genéricas `<div>`. Um código bem estruturado é a base para que os leitores de tela entendam o que está acontecendo na página.

A **navegação por teclado** é essencial não apenas para pessoas com deficiências visuais que usam leitores de tela, mas também para usuários com mobilidade reduzida, tendinites ou até mesmo usuários avançados (power users) que preferem a agilidade das teclas.

Para garantir que a navegação por teclado funcione corretamente, a ordem do código **HTML** deve ser lógica. O fluxo do documento (Document Object Model - **DOM**) **dita como** a tecla **Tab** **percorre os elementos interativos**, como links (`<a>`), botões (`<button>`) e campos de formulário (`<input>`). O foco visual (aquela borda que aparece ao redor do elemento selecionado) jamais deve ser removido com CSS (`outline: none;`) sem que um substituto acessível seja providenciado.

As **Diretrizes de Acessibilidade para Conteúdo Web (WCAG)** são o padrão global para garantir que sites, aplicativos e documentos digitais sejam acessíveis a pessoas com deficiência. Elas são **organizadas de forma cumulativa**, o que significa que **para atingir um nível mais alto, você precisa obrigatoriamente cumprir todos os requisitos dos níveis anteriores**.

Aqui está o detalhamento prático do que cada um desses níveis significa no dia a dia do desenvolvimento e design web:

Nível A: Requisitos básicos (Essencial)

Este é o nível de sobrevivência da acessibilidade. Se um site não atende aos critérios do Nível A, ele é efetivamente impossível de ser usado por um grupo significativo de pessoas com deficiência.

- O que ele exige: Soluções para as barreiras mais críticas.
- Exemplos práticos:
 - Fornecer texto alternativo (a tag `alt`) para imagens, para que leitores de tela possam descrevê-las para pessoas cegas.
 - Garantir que o site possa ser navegado usando apenas o teclado (sem depender do mouse).
 - Não usar cor como o único meio de transmitir uma informação (ex: "clique no botão vermelho para cancelar").
 - Fornecer legendas para vídeos pré-gravados.

Nível AA: O padrão de mercado (Ideal para a maioria dos sites)

Este é o "ponto de equilíbrio" e a meta que a grande maioria das empresas e governos adota. Atingir o Nível AA significa que seu site é acessível para a maioria das pessoas, abrangendo uma ampla gama de deficiências físicas, visuais, auditivas e cognitivas. Hoje, é considerado o requisito legal básico em muitos países e na maioria das políticas corporativas.

- O que ele exige: Barreiras removidas, com um foco maior na usabilidade confortável e flexibilidade para o usuário. (Lembre-se: inclui tudo do Nível A).
- Exemplos práticos:
 - Contraste de cor: O texto deve ter um contraste mínimo (geralmente 4.5:1) em relação ao fundo para ser lido confortavelmente por pessoas com baixa visão.
 - O site deve continuar legível e funcional se o usuário aplicar um zoom de até 200%.
 - Navegação consistente: os menus e a estrutura do site devem se repetir de forma previsível em todas as páginas.
 - Indicadores visíveis de foco (uma borda ao redor de links e botões quando navegados pelo teclado).

Nível AAA: Nível máximo (Especializado)

Este é o padrão de excelência (Padrão Ouro). É o nível mais rigoroso e abrangente. A própria W3C (organização que cria a WCAG) alerta que não é recomendado exigir conformidade AAA para sites inteiros, pois muitas vezes é inviável tecnicamente ou compromete severamente o design para certos tipos de conteúdo.

- O que ele exige: Melhorias que atendem a deficiências muito específicas, maximizando a clareza e o suporte técnico.
- Exemplos práticos:
 - Contraste de cor ainda mais alto (mínimo de 7:1).
 - Tradução de vídeos e áudios em Linguagem de Sinais (como Libras no Brasil).
 - Textos escritos em linguagem extremamente simplificada para pessoas com deficiências cognitivas ou de leitura, com glossários para jargões.
 - Nenhum limite de tempo para preencher formulários ou concluir tarefas.

Resumo: Comece garantindo o Nível A para que ninguém seja completamente bloqueado, mire e estabeleça o Nível AA como sua meta principal de lançamento, e reserve o Nível AAA para seções específicas do seu site que demandem o máximo de inclusão (como portais de serviços públicos especializados).

D. Explicação técnica:

A navegação por teclado **depende** intrinsecamente **do foco** (:focus no CSS) e da **ordem do DOM**. Elementos nativos do **HTML5**, como `<button>` e `<a href="...">`, já possuem comportamento de foco automático pelo navegador. **Elementos não interativos**, como `<div>` ou `<span>`, **não recebem foco**. Se for **estritamente necessário** tornar uma `<div>` interativa (o que **não é recomendado**), deve-se **utilizar o atributo** `tabindex="0"` para incluí-la na ordem de tabulação. O `tabindex="-1"` remove um elemento nativo da tabulação, mas ainda permite que ele receba foco via JavaScript. A propriedade CSS `outline` é a responsável por dar o feedback visual de onde o usuário está no momento.

E. Exemplo de Código:

HTML

```
<div class="botao-falso" onclick="enviar()">Enviar</div>
```

```
<button class="botao-real" onclick="enviar()">Enviar</button>
```

```
<style>
```

```
/* NUNCA remova o outline sem prover uma alternativa */
```

```
.botao-real:focus {  
  /* Melhorando a visualização do foco para acessibilidade */  
  outline: 3px solid #0056b3;  
  outline-offset: 2px;  
}  
</style>
```

F. Estudo de Caso: Uma grande varejista criou um modal de promoções lindo, mas esqueceu de "prender" o foco **dentro do modal**. Usuários de leitores de tela abriam o modal, **apertavam a tecla Tab**, e o foco ia para os elementos **atrás do modal** (na página obscurecida), tornando **impossível fechar a propaganda ou continuar a navegação**. A solução foi criar uma armadilha de foco (**Focus Trap**) via JavaScript para garantir que o usuário só navegasse dentro do modal até fechá-lo.

G. Exercícios:

1. Abra a página inicial do seu projeto. Desconecte (ou ignore) o seu mouse. Tente navegar por todos os links e botões da página utilizando apenas a tecla **Tab**. Anote os elementos que você não conseguiu acessar.
2. Inspecione o CSS do seu projeto. Busque por declarações de **outline: none** ou **outline: 0**. Se existirem, substitua-as por estilos de foco personalizados que mantenham a identidade visual da marca, mas garantam alta visibilidade.
3. Adicione a pseudo-classe **:focus-visible** aos seus botões e links e observe a diferença de comportamento em relação ao **:focus** tradicional ao clicar com o mouse versus navegar com o teclado.

Conceito 2: Atributos ARIA e o atributo **alt**

A. Pergunta: Se uma imagem no seu site não carregar devido a problemas de rede, ou se um deficiente visual acessar seu site com um leitor de tela, como ele saberá qual é o conteúdo daquela imagem?

B. Cenário Real:

Desenvolvedores frequentemente criam menus *dropdown* complexos ou *switches* (botões de liga/desliga) estilizados com `<div>` e CSS. Para um leitor de tela usado por um deficiente visual, aquilo é apenas texto vazio e não interativo. O usuário cego não saberá se o botão está ligado ou desligado, gerando uma barreira crítica na utilização do software.

C. Explicação teórica:

O **HTML5 possui semântica própria**, mas muitas vezes a criatividade do design exige componentes de interface que não têm uma tag HTML nativa equivalente (como um *slider* complexo ou um sistema de abas). É aqui que entra o **WAI-ARIA (Web Accessibility Initiative - Accessible Rich Internet Applications)**. O **ARIA** não muda o comportamento ou o visual de um elemento, ele apenas fornece metadados (informações extras) para tecnologias assistivas, traduzindo o que aquele elemento customizado significa.

O atributo mais básico e vital da **acessibilidade** é o **alt (alternative text)** em imagens. O texto alternativo tem um **duplo propósito**: ele é **lido em voz alta** por leitores de tela para usuários com deficiência visual e é **exibido na tela se a imagem quebrar** ou não puder ser carregada. Sem o atributo

`alt`, o leitor de tela muitas vezes lê o nome do arquivo da imagem (ex: `IMG_2023_final_v2.jpg`), o que causa enorme confusão e frustração para o usuário.

Além disso, os atributos **ARIA**, como `aria-label`, `aria-hidden` e `aria-expanded`, ajudam a dar contexto. **Por exemplo**, um botão que possui apenas um ícone de lupa (sem texto) não fará sentido para o leitor de tela. O uso de `aria-label="Buscar"` resolve o problema, garantindo que o propósito do botão seja perfeitamente inteligível para o software de acessibilidade sem poluir o visual da página.

D. Explicação técnica:

A regra de ouro do ARIA é: **O melhor ARIA é não usar ARIA**. Sempre prefira as **tags semânticas** do HTML5 (`<nav>`, `<header>`, `<main>`, `<button>`). Use **atributos ARIA apenas para preencher lacunas** de semântica. O `aria-hidden="true"` esconde um elemento do leitor de tela (útil para ícones decorativos). O `alt=""` (vazio) diz ao leitor de tela para ignorar a imagem (útil para imagens puramente decorativas). Já o `aria-expanded` indica se um menu sanfona (accordion) está aberto (`true`) ou fechado (`false`).

E. Exemplo de Código:

```
HTML




<button aria-label="Fechar janela" onclick="fecharModal()">
  <i class="fas fa-times" aria-hidden="true"></i>
</button>
```

F. Estudo de Caso: Uma prefeitura lançou um portal para agendamento de consultas. Os horários disponíveis eram marcados por quadrados verdes e os indisponíveis por quadrados vermelhos (usando `<div>`). Pessoas cegas não enxergam cores. Sem texto alternativo ou atributos ARIA informando o status (`aria-disabled="true"`), os usuários não conseguiam diferenciar os horários. A correção envolveu adicionar atributos ARIA e textos visuais apenas para leitores de tela (`sr-only`), resolvendo o problema de inclusão.

G. Exercícios:

- Revise todas as tags `<img>` do seu projeto. Garanta que todas possuam o atributo `alt`. Imagens relevantes devem ter descrições concisas; imagens de fundo ou decorativas devem ter `alt=""`.
- Identifique todos os botões do seu sistema que contêm apenas ícones (ex: botão de lixeira para excluir, lupa de busca). Adicione o atributo `aria-label` apropriado a eles.
- Aplique a classe `aria-hidden="true"` a todos os ícones da sua interface (seja FontAwesome, SVG ou imagens decorativas) para evitar que o leitor de tela leia o código gerado pelo ícone.

Conceito 3: Auditoria com Lighthouse

A. Pergunta: Como podemos provar para o nosso gerente de projetos (que nosso código segue as regras de acessibilidade sem precisarmos adivinhar ou testar manualmente tudo?

B. Cenário Real:

Durante o Sprint de entrega, a **equipe de QA** (Quality Assurance) **reprovou o deploy** da aplicação porque **não cumpria** o índice mínimo de **90% em acessibilidade** exigido no contrato do cliente. O desenvolvedor **front-end** precisava de uma **ferramenta rápida** e integrada ao navegador para gerar relatórios, mapear os gargalos e atestar que suas correções resolveram o problema.

C. Explicação teórica:

O **Lighthouse** é uma ferramenta automatizada de código aberto incorporada ao **Chrome DevTools**, projetada para melhorar a qualidade das páginas da web. Ele executa uma **bateria de testes** contra a página renderizada e gera um relatório abrangente sobre **Desempenho, Acessibilidade, Melhores Práticas e SEO**. No pilar de Acessibilidade, o Lighthouse **varre o DOM** da página em busca de **violações das regras WCAG**, como contraste de cor insuficiente, falta de rótulos (**labels**) em formulários e hierarquia incorreta de cabeçalhos (**<h1>** a **<h6>**).

Auditar o software de forma automatizada garante agilidade no processo de desenvolvimento e está intimamente ligado à capacidade de aplicar boas práticas e garantir a qualidade do produto final. O relatório não apenas aponta o erro, mas **indica exatamente em qual linha de código ele se encontra** e fornece um link para a documentação explicando como corrigi-lo.

Apesar de poderoso, o Lighthouse **não detecta 100%** dos problemas de acessibilidade, pois algoritmos não conseguem julgar o **"contexto"** de um texto alternativo (se o **alt** fizer sentido ou não, ele apenas checa se existe). Portanto, a avaliação com a ferramenta deve sempre ser complementada por testes manuais, como o *code review* em pares com *checklists* de acessibilidade.

D. Explicação técnica:

Para executar o Lighthouse, o desenvolvedor deve abrir o **Chrome DevTools (F12)**, navegar até a aba **"Lighthouse"**, selecionar a categoria **"Acessibilidade" (Accessibility)** e o dispositivo de teste (**Mobile ou Desktop**) e clicar em **"Analyze page load"**. A ferramenta irá simular o carregamento, analisar a árvore de acessibilidade do navegador e retornar uma nota de 0 a 100. Problemas são divididos em níveis de gravidade: **Crítico, Sério, Moderado e Leve**. É obrigatório corrigir primeiro os itens críticos (falhas graves de usabilidade).

E. Estudo de Caso: A refatoração do Dashboard. A equipe estava com uma nota 65 no Lighthouse. A análise apontou três problemas: botões pequenos demais em dispositivos móveis (falta de área de toque), contraste baixo entre o texto cinza claro e o fundo branco, e uso excessivo da tag **<h1>**. Em apenas 30 minutos, a equipe corrigiu o padding dos botões, escureceu a cor da fonte (**#333**) e ajustou a semântica para **<h2>** e **<h3>**. A nova auditoria resultou em nota 98, habilitando o projeto para o pitch final.

F. Exercícios:

1. Abra a página principal do seu projeto e rode o Google Lighthouse com foco exclusivo na categoria "Accessibility".
2. Salve o relatório inicial. Identifique os três piores erros listados.
3. No painel, clique em "Learn more" (Saiba mais) ao lado de um erro que você desconheça. Leia a documentação sugerida para entender o porquê daquele erro existir.

3. FAQ - Dúvidas Reais

1. O Lighthouse atesta se o meu texto alternativo (**alt**) é bom?

Resposta: Não. O Lighthouse apenas verifica se o atributo **alt** está presente na tag ****. Ele não compreende o contexto para saber se a sua descrição "foto de uma mulher sorrindo segurando um

notebook" faz sentido ou se você apenas digitou `alt="1234"`. A qualidade semântica exige revisão humana.

2. Posso usar `aria-label` em uma `<div>` genérica para transformá-la num botão?

Resposta: Embora seja tecnicamente possível aplicar atributos ARIA a uma `<div>`, essa é uma má prática. O ideal é usar a tag semântica `<button>`. Se usar uma `<div>`, você precisará recriar todo o comportamento nativo (foco de teclado com `tabindex`, ativação por `Enter` e `Space` via JavaScript) manualmente.

3. O que acontece se eu esquecer de colocar uma `<label>` para meu `<input>`?

Resposta: Usuários de leitores de tela não saberão o que digitar naquele campo. Além disso, a tag `<label>` melhora a acessibilidade para pessoas com dificuldade motora, pois clicar na palavra da *label* automaticamente foca o campo de *input* ou marca o respectivo *checkbox/radio*.

4. Minha equipe fez um sistema com tema escuro (Dark Mode). Preciso me preocupar com contraste?

Resposta: Muito! Temas escuros frequentemente sofrem com baixo contraste se forem usadas fontes em tons de cinza escuro ou botões escuros sem borda clara. O Lighthouse apontará falhas caso o contraste (ratio) entre o texto e o fundo seja menor que 4.5:1 para textos normais.

5. Qual a diferença entre esconder elementos com `display: none` e `aria-hidden="true"`?

Resposta: O `display: none` oculta o elemento visualmente na tela E também o esconde do leitor de tela (remove da árvore de acessibilidade). Já o `aria-hidden="true"` mantém o elemento visível na tela para usuários videntes, mas faz com que as tecnologias assistivas (leitores de tela) ignorem completamente sua existência.

4. Atividade Prática Guiada

Contexto do problema: A equipe precisa garantir que os formulários e a interface desenvolvidos até agora estejam prontos para o *pitch* com os investidores, seguindo rigorosos padrões de qualidade e documentação.

Requisitos: Executar uma auditoria do projeto utilizando a ferramenta Lighthouse.

- Realizar um *Code Review* focado em acessibilidade, utilizando as métricas WCAG 2.1.
- Identificar e corrigir no mínimo 3 (três) problemas de acessibilidade reais no código.
- Gerar um pequeno relatório demonstrando o "Antes" e o "Depois".

Sugestões de solução:

Focar primeiro nos erros mais críticos apontados pela ferramenta, geralmente ligados a contraste, navegação por teclado, rótulos de formulário ou falta de textos alternativos.

Passo a passo liderado pelo Docente (Mão na Massa):

1. **Auditoria Inicial:** O professor abrirá um projeto modelo (com falhas propositalmente) no navegador e acionará o Lighthouse (F12 > Lighthouse > Accessibility > Analyze).
2. **Análise de Resultados:** O docente lerá os resultados com a turma. *Exemplo:* "A nota foi 72. Temos um erro de contraste, imagens sem alt e inputs órfãos."
3. **Correção ao vivo (Code Review):** - Passo A (Contraste): Abrir o `style.css` e ajustar a cor baseando-se no *color picker* do DevTools, que mostra a linha de contraste segura.
 - Passo B (Formulário): Abrir o `form-crud.html`, associar a tag `<label for="cpf">` com o `<input id="cpf">`.

- Passo C (Imagens e ARIA): Adicionar `alt` descritivo nas imagens. Adicionar `aria-label="Deletar Registro"` no botão de ação da tabela.
4. **Re-auditoria:** O professor rodará o Lighthouse novamente, mostrando a nota subindo para 100 e validará usando a tecla `Tab`.

Resultado esperado: Um relatório exportado pelo Lighthouse comprovando a melhora da nota e os commits no GitHub demonstrando as correções semânticas aplicadas na interface.

5. Exercício: Auditoria nos Sites Senai SC

Abra os seguintes sites do Sesi/Senai abaixo:

<https://sesisc.org.br>
<https://sc.senai.br>
<https://estudante.sesisenai.org.br>
<https://ava.sesisenai.org.br>

e rode o **Lighthouse(F12 do Chrome)** e o site accessmonitor.acessibilidade.gov.pt

Identifique 3 falhas de acessibilidade - para cegos ou surdos - e explique como você as corrigiria usando código.

DICA: teste também mais duas URLs específicas de cada site.

Ex.: <https://estudante.sesisenai.org.br/documentos>

Faremos um relatório coletivo em sala a ser entregue ao SENAI SC apontando os principais pontos de melhoria.

6. Conexão com o Curso

Aplicação na Situação de Aprendizagem: O projeto de vocês será avaliado por uma banca e deve possuir a capacidade de escalar para usuários reais no mercado de startups. Investidores não aportam capital em plataformas que limitam sua base de clientes ou expõem a empresa a processos de quebra de diretrizes de acessibilidade (como a Lei Brasileira de Inclusão). Vocês acabaram de garantir que o produto digital atende perfeitamente ao Requisito Não Funcional (RNF) de Usabilidade, alinhado com a Capacidade Técnica (CT01). Todas as correções feitas e atestadas pelo relatório do Lighthouse hoje servem de documentação comprobatória da qualidade do software de vocês.

Próxima Aula: Com a acessibilidade validada e a estrutura construída, vamos para a reta final do front-end na nossa **Aula 13 - Revisão Frontend!** Vamos integrar todas essas páginas num fluxo de navegação completo (`Login -> Dashboard -> Lista -> Formulário`), faremos revisões entre duplas e consolidaremos o versionamento no GitHub, deixando tudo pronto para nossa avaliação. Preparem-se para o fechamento do Bloco 02!